# Security Audit Report

pondora-contracts

| | |
|---|---|
| **Date** | November 19, 2025 |
| **Project** | pondora-contracts |
| **Version** | 2.2.0 |

# Contents

# Disclosure

This document contains proprietary information belonging to No Witness Labs. Duplication, redistribution, or use, in whole or in part, in any form, requires prior written consent from No Witness Labs.

Nonetheless, both the customer **pondora-org** and No Witness Labs are authorized to share this document with the public to demonstrate security compliance and transparency regarding the outcomes of the Protocol.

# Disclaimer

This review reflects the state of the codebase at the time of assessment and should be regarded as a point-in-time evaluation. Any changes made after the assessment period fall outside the scope of this report.

Although every effort was made to identify potential risks and vulnerabilities, no security review can guarantee the discovery of all issues. The information and recommendations in this report are provided solely for technical purposes and must not be considered investment advice.

To strengthen security assurance, projects are encouraged to seek multiple independent reviews and to supplement audits with practices such as bug bounty programs.

The scope of this audit was limited to the code provided. It did not extend to compiler-level artifacts (e.g., generated UPLC code) or to areas beyond the explicitly reviewed codebase. Additional testing methodologies, including unit testing and property-based testing, were also outside the scope of this engagement.

### *Temporary owner privileges and authorization trust*

The temporary owner mechanism grants a delegated key (`temporary_public_key`) broad authorization capabilities over the smart account for the duration of its validity period (`valid_until`). During this window, the temporary owner can perform any intent-related action that the primary owner key could, including the creation, execution, and destruction of intents.

This audit assumes that all temporary owner delegations are intentional, correctly authorized, and that the authorizing signature (`authorizing_public_key` → `temporary_public_key`, `valid_until`) is securely managed off-chain. Authorization mistakes, misuse of delegated keys, or exposure of private keys fall outside the scope of this review. The framework imposes no on-chain restrictions or scoping beyond the expiry timestamp, and security during the delegation period depends entirely on the correctness of the off-chain authorization process.

### *Reliance on off-chain intent correctness*

The protocol relies critically on the correctness of Intents, which encapsulate the module to be executed, its arguments, the amount to spend from the smart account, and other execution parameters.

On-chain logic verifies only signature-level integrity, without validating the structure, semantics, or intended behavior of the data provided. When an intent is published, its contents are not validated or inspected on-chain; validators perform no checks on the intent's structure, fields, or semantic correctness.

This audit assumes that all intents are accurately constructed and authorized off-chain, reflecting the true intentions of the account owner or authorized delegate. If incorrect, malformed, or malicious intents are generated off-chain, the validators will accept and execute them as valid, potentially leading to unintended asset movements or module calls. Such scenarios fall outside the scope of this review and are the responsibility of the off-chain intent management layer.

### *Trust boundary between core protocol and external modules*

The protocol framework is designed to allow any compatible module to be executed by a Pond smart account. Account security therefore depends not only on the correctness of the core validators but also on the security and integrity of the individual modules they execute. This audit covers only the reviewed modules and the base protocol logic; it does not guarantee protection against vulnerabilities introduced by future or unaudited modules. Malicious or incorrectly implemented modules could expose the protocol to security risks or unintended asset flows beyond the scope of this review.

### *Scope and flexibility of the Child Intents system*

The Child Intents mechanism is intentionally general and designed for flexible orchestration between parent and child intent sets. Parent intents may create, destroy, or reuse multiple child roots, and modules can freely define the conditions under which these actions occur. The protocol does not enforce restrictions on multiplicity, repetition, or lifecycle; such constraints are expected to be implemented within the respective parent modules.

As a result, the security and correctness of the Child Intents system depend heavily on the modules that implement or leverage it, as well as on the off-chain logic that constructs and manages these relationships. This audit covers only the generic framework and cannot guarantee protection against misuse, unintended mint/burn loops, or logic flaws introduced by future modules or strategies built on top of the Child Intents system.

### *Interactions with external protocols*

Certain modules—such as `module_unlock_fungible_outputs_for_payment` and any future extensions—interact directly with external protocols and on-chain systems. The correctness, safety, and economic outcomes of such interactions depend on the behavior and security of those external protocols. The Pondora framework does not enforce or verify the integrity, availability, or correctness of external protocols it interacts with.

This audit assesses only the internal logic of the reviewed Pondora modules and validators. Any risks or failures originating from external protocols, liquidity sources, or integrations lie outside the scope of this review and may directly affect the safety of dependent modules or user funds.

# Scope

**In scope**

- On-chain validators and supporting logic at the time of review.
- Datum/redeemer structures and on-chain invariants (value, units/rates, cross-phase checks).
- Use of reference UTxOs for configuration and script parameters.

**Out of scope**

- Compiler layer / UPLC output and toolchain internals.
- Creation of new unit tests or property-based tests.
- Off-chain libraries, front-end apps, wallet UX (incl. blind-signing), unless explicitly constrained on-chain.
- Third-party protocol internals (bridges/liquidity venues), centralized services, exchanges, oracles, external price feeds.
- Node operations, networking, mempool policy variance, ordering/MEV-like effects.

# Assessment Overview

From **September 15**, **2025** to **November 14**, **2025**, **pondora-org** engaged No Witness Labs to conduct a comprehensive security assessment of the **pondora-contracts** protocol. The evaluation was performed in accordance with established industry best practices for code auditing and protocol analysis.

The audit followed a structured five-phase process:

- **Planning** – Defined the assessment scope and objectives, and aligned expectations with the client.
- **Discovery** – Acquired an understanding of the protocol through client discussions, documentation review, and preliminary analysis of architecture and design.
- **Manual Revision** – Conducted an in-depth review of the codebase to identify potential vulnerabilities, design weaknesses, and security risks.
- **Validation** – Verified client-provided fixes and obtained acknowledgments for resolved issues.
- **Reporting** – Documented all confirmed findings, assigned severity levels, and provided actionable recommendations for remediation.

The team also evaluated protocol efficiency and potential optimizations. Findings were prioritized by severity level.

# Assessment Components

**Manual revision**

Our manual code auditing is focused on a wide range of attack vectors, including but not limited to:

- UTXO Value Size Spam (Token Dust Attack)
- Large Datum or Unbounded Protocol Datum
- EUTXO Concurrency DoS
- Unauthorized Data modification
- Multisig PK Attack
- Infinite Mint
- Incorrect Parameterized Scripts
- Other Redeemer
- Other Token Name
- Arbitrary UTXO Datum
- Unbounded protocol value
- Foreign UTXO tokens
- Double or Multiple satisfaction
- Locked Ada
- Locked non Ada values
- Missing UTXO authentication
- UTXO contention

# Executive Summary

## Project Overview

Pondora is a smart account framework that allows users to authorize and orchestrate DeFi actions (intents) against assets held at a Pond address.

The Pond script ensures a specific Pond variant is active and defers validation to that variant's withdrawal logic.

Business logic is implemented by module validators; intents reference a specific module and its parameters to express what must be enforced.

Intents are referenced via a Merkle Patricia Trie (MPT); executors prove membership and satisfy module-specific rules to unlock assets.

Labels classify Pond UTxOs and guide intent selection: the redeemer supplies a label plus a LabelType (Datum/OutRef/Unit/Ada) indicating the derivation scheme.

Child Intents allow owners to activate or retire a separate MPT root when a parent intent is satisfied, enabling on-chain automation.

The system's building blocks include:

- Pond (spend) with Pond Variants (withdrawals) for input-level checks and signature/authorization enforcement.
- Intents & Child Intents authorized on-chain (intent token + root in datum) or off-chain (signed message).
- Pond Modules (withdrawal validators) that implement concrete business logic (payments, defragmentation, dependencies).
- Accounts/Change pattern to ensure fungible spends are repaid.

# Extended Overview

### Design Overview

- Smart-account model: Pond spends are permitted only when the active Pond variant also appears in withdrawals; that variant then validates each relevant input, MPT proof(s), module presence, and any required signatories or on-chain intent references.
- Intent authorization: Owners or an authorized TemporaryOwner can (a) keep a reference UTxO that holds an intent token (minted under the Intents Validator policy) whose datum carries the authorized MPT root; or (b) authorize off-chain with a signed message;
- Modules and flows: A transaction may satisfy one or more module-specific intents per spent UTxO. For fungibles, modules following the accounts pattern compute precise change and bind it to a tag derived from spent outrefs to prevent double satisfaction.

### Key Characteristics

- MPT-based, batched authorization of many intents.
- Off-chain authorization via signed intents.
- Optional authority delegation via temporary owner for session UX.
- Exact-change pattern for fungible assets (accounts/tagging) where modules adopt it.

### Roles and Responsibilities

- Owner: Authorizes intents (on-chain token or signed message) and may delegate temporarily for sessions.
- Executor (third-party): Builds transactions that prove MPT membership and satisfy module-specific rules; earns fees where applicable.
- Helpers/Certificates: Some flows require stake registration (nonces) to enforce single-use guarantees.

### Lifecycle

1. Authorize intents: Owner mints an intent token with the MPT root in datum, or signs an off-chain message (may include a TemporaryOwner session).
2. Execute: A third party assembles a tx, including the Pond variant withdrawal, required module withdrawals, and proofs for each intent.
3. Accounts settle: For fungibles, exact change is enforced to a tagged output at the Pond address; nonces/certificates validated if required.

### Supporting Modules

- Defragment: Consolidate fragmented balances; repay fungibles fully.
- Dependencies (Fee): Unlock a specified amount (often ADA) only when a set of other intents is present in the same transaction; useful to incentivize processing.
- Unlock for Payment: Single-use (nonce-gated) payment with valid_from/valid_to window; exact datum/address/value match (ADA or token + optional lovelace).

# Code Base

## Repository

https://github.com/pondora-org/pondora-contracts-audit.git

## Commit

`734d8fe664a1c17f8373c74cbaa990a51ff3e842`

## Files Audited

| Files | SHA256 Checksum |
|---|---|
| lib/pondora/accounts.ak | `7647312a27781c80b1ddc1e1a3f1b88953ab5fe53a11b285152b72db43ea79b6` |
| lib/pondora/messages.ak | `44a78eef54c1ac5b51bee06b732cf6965586d750dc2a11beac068354cdb065f4` |
| lib/pondora/types.ak | `e07b8b8d98aef57090cb930948b90fbc4bf3903268c4ad3704c2279537bd01dc` |
| lib/pondora/utils.ak | `41541f968610298219d682a344b80a599bd92de11782ac438e0d68a4beff3f7d` |
| validators/pond.ak | `204437f94facfe9f2d08a12568332124564347 86fb176c24f703af92b3df7826` |
| validators/pond_native.ak | `cd1da63ff9bae852926312dad4a2e239ce79585d32558609f9bd9f3c8fb31e57` |
| validators/intents_native.ak | `26934b7401d205673f1b1234e2568b068c3c200ac60f40e6b752da5a8d960009` |
| validators/modules/helper_nonce.ak | `351a816aa4c124002c246fed6b529255744b8d8f110b80d3930457e0c6a28a42` |
| validators/modules/module_defragment.ak | `ac3c6ff32d17e4b41b1372b4aa99dfc4913714a408bb5c5ffad03e5577e1a76e` |
| validators/modules/module_dependencies.ak | `a83e3a5eac931cb209ad21072aa6fd3a278222f4d38b8398a987178d0c548892` |
| validators/modules/module_unlock_fungible_outputs_for_payment.ak | `e82133c41a16ac74a3577a18240f322661afc3a577eaf9b93b1ad772ba8f04f7` |

# Severity Classification

- **Critical**: Vulnerabilities that enable direct theft of assets, permanent loss of funds, or catastrophic protocol failure. These represent immediate and severe financial risk with straightforward exploitation paths.
- **High**: Vulnerabilities that can result in financial loss or significant harm to users or the protocol, though impact may be limited to specific functions or require certain conditions. Exploitation paths exist but may be more complex than Critical Vulnerabilities.
- **Medium**: Vulnerabilities that do not directly result in financial loss but may temporarily impair functionality or create exploitable conditions. These are typically limited to state manipulations and include issues such as front-running susceptibility or logic flaws that compromise transaction integrity.
- **Low**: Low-impact Vulnerabilities with limited attack vectors, minimal attacker incentive, or requiring highly specific scenarios. These typically do not result in financial loss or significant harm to users or the protocol.
- **Informational**: Findings without immediate security or financial implications. These include optimization opportunities, code quality improvements, inconsistencies in naming conventions, design suggestions, and gaps in documentation.

# Finding Severity Ratings

The following table defines levels of severity and score range that are used throughout the document to assess vulnerability and risk impact

| Level | Severity | Status |
|---|---|---|
| 5 | Critical | 0 |
| 4 | High | 1 |
| 3 | Medium | 5 |
| 2 | Low | 4 |
| 1 | Informational | 18 |

# Findings

| ID | Title | Severity | Status |
|---|---|---|---|
| 401 | Invalid Nonce Script Possible | High | Mitigated |
| 301 | Malicious Module / Intent Swap | Medium | Acknowledged |
| 302 | Temporary Owner Delegation Has No Revocation Path | Medium | Acknowledged |
| 303 | Unclaimable Protocol Fees from Long-Lived Nonce Intents | Medium | Acknowledged |
| 304 | LabelOutRef Wildcard Intent Theft & Risks | Medium | Acknowledged |
| 305 | DDOS of Smart Account Funds via Defragmentation | Medium | Acknowledged |
| 201 | Optimize Defragment Module's Redemptions | Low | Resolved |
| 202 | Duplicate Root Hash Breaks Intent Revocation Guarantees | Low | Mitigated |
| 203 | Unverified `label` / `quantity` in ByModule ( `LabelDatum` ) | Low | Mitigated |
| 204 | Expired Intents Can Be Executed | Low | Resolved |
| 101 | Use Scott Encoding | Informational | Resolved |
| 102 | Optimize List Iteration | Informational | Resolved |
| 103 | Optimize Dict Iteration | Informational | Resolved |
| 104 | Optimize `ChangeDict` Creation | Informational | Resolved |

| ID | Title | Severity | Status |
|---|---|---|---|
| 105 | Redundant Key Field in `NativeAuth` Structure | Informational | Resolved |
| 106 | Optimize Iteration Over Sorted Pairs | Informational | Resolved |
| 107 | Redundant Validation in Spend Path During Burn/Mint | Informational | Resolved |
| 108 | Variant-Level `pond_script` Validation Implemented per Module | Informational | Acknowledged |
| 109 | Redundant Range Handling and Comparison Logic in `must_be_before` | Informational | Resolved |
| 110 | Redundant `ModuleArgs` Fields and Checks | Informational | Resolved |
| 111 | `nonce_script` Should Use `Option<ScriptHash>` | Informational | Resolved |
| 112 | Remove Redundant Owner Checks from Modules | Informational | Resolved |
| 113 | Spam UTxO / Token Pollution | Informational | Mitigated |
| 114 | ChildRef Root Index Optimization | Informational | Resolved |
| 115 | Incorrect Comment on `valid_from` Check | Informational | Resolved |
| 116 | Redundant `owner` Field in Uniqueness Tuple | Informational | Resolved |
| 117 | Miscellaneous Optimizations | Informational | Resolved |
| 118 | Optimize Required Intents Serialization | Informational | Resolved |

## [H 401] Invalid Nonce Script Possible

| Level | Severity | Status |
|---|---|---|
| 4 | High | Mitigated |

**Category:** Replay Protection / Off-Chain Trust Assumptions

**Affected Components:** `helper_nonce` instantiation & parameter binding

### Description

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

By specifying a nonce script hash in an intent, the protocol ensures that it is executed just once until it is valid. This is enforced by requiring the nonce withdrawal script to be registered in the transaction that executes the intent. Since the de-registration of this script is possible only after `intent_valid_until`, it is ensured that the nonce script is registered only once. It is crucial that the nonce script hash is obtained by applying correct parameters (`intent_owner`, `intent_valid_until`, etc) to the `helper` validator. If incorrect parameters are applied or an incorrect script is used, it allows an intent to be executed multiple times, which could potentially result in financial losses. The `pond_native` validator fails to ensure that the intent's nonce script hash is a valid instance of the `helper` validator. As a consequence, the user implicitly trusts the off-chain library to set a valid nonce script hash. This puts user funds at the mercy of an off-chain library, which carries risks.

### Recommendation

On-chain Verification of Parameter Application: `pond_native` validator verifies that the nonce script hash is valid by computing it on-chain using the unapplied `helper` validator and script parameters received via redeemer. This provides a trustless path to only ever execute intents with a valid nonce script hash. However, this approach is substantially expensive. There is a CIP proposed to provide a built-in function for making this validation efficient and cheaper.

Inlining the implementation details without the built-in from the above CIP:

1. The whole target contract has to be wrapped within an outer function to which the parameters will be applied. This leads to single occurrences of the parameters throughout the script's CBOR.
2. Each parameter will be required to have fixed lengths to allow validation of the bytes surrounding parameters. This can be achieved by assuming these parameters are all hashes and that their corresponding values will be provided via the redeemer.
3. To validate a given script is an instance of a known script, the first bytes of an instance must be provided (up to where the parameters are placed). With this "prefix" at hand, the instance's CBOR can be constructed on-chain as such:

```
1 let appliedCBOR =                                                    Plutus
2   prefix <> param0 <> paramHeader <> param1 <> paramHeader <> param2 <> postfix
```

Where postfix is similar to prefix, but for the rest of the script until its end. Note that `paramHeader` can be reused, assuming all parameters have equal lengths.

4. This applied CBOR can be hashed to attain its corresponding script credential.

Since the significant costs of on-chain nonce script verification can render the above trustless solution infeasible due to its high costs, we propose an alternative. It is designed to facilitate an intuitive user interface where intents and their fields, particularly the nonce script hash, can be reviewed and validated by users. While this solution still requires trusting the protocol interface, the trust requirement is made explicit.

**Resolution**
Mitigated off-chain by requiring and facilitating user inspection of intents for correctness. Resolved in commit hash `8a67a19c4761ed2d1933e682f00cc075630c73ba`

## [M 301] Malicious Module / Intent Swap

| Level | Severity | Status |
|---|---|---|
| 3 | Medium | Acknowledged |

**Category:** Authorization Model / Module Trust

**Affected Components:** Intent validation, module execution, temporary ownership system

**Description**

An attacker can deploy a malicious Pondora module, create an intent referencing it, and then trick the user into authorizing that intent. Once authorized, the malicious module can drain UTxOs and assume control of the entire smart account.

A secondary route involves TemporaryOwner abuse: if an attacker obtains or is granted temporary ownership, they can authorize or mint intents that point to malicious modules — yielding the same full-account compromise.

**Attack Steps**

1. Deploy a malicious module: The attacker creates a malicious Pondora module.
2. Create intent referencing it: The attacker constructs an intent pointing to that module's script hash.
3. Trick the owner into signing it: The attacker deceives the owner (e.g., through UI/UX manipulation) into authorizing the malicious intent.
   - *(Alternative path)*: the attacker becomes or is granted the `TemporaryOwner`, gaining the right to authorize or mint intents referencing arbitrary modules.

*Background: Intents include a `module` field referencing an external withdrawal script responsible for executing redemptions. These modules define the specific logic an authorized intent can perform.*

**Why the Impact is critical**

- This vulnerability exposes the entire smart account to compromise, not just the transaction in which it occurs.
- Once a malicious module is authorized—either directly or through a temporary owner—it can execute arbitrary actions across all UTxOs controlled by that account.
- Even if the user later revokes or replaces the intent, the attacker may still retain control or re-establish it through mechanisms like the child-intent system, effectively making recovery uncertain and highly complex.

**Impact**

- Full-account compromise: Once authorized, the malicious module can perform arbitrary actions across all UTxOs under the smart account.
- Permanent loss of assets: The attacker can drain all ADA/tokens.
- Persistent control via child intents: A malicious module may reassert access through derived or chained intents.

**Root Causes**

- The `module` field in `Intent` allows arbitrary external script hashes.
- There is no mechanism ensuring the referenced module is trusted, audited, or protocol-approved.
- The authorization process assumes the owner can safely distinguish and trust modules — an unsafe assumption in real-world UX.
- TemporaryOwner accounts can authorize new intents/modules without sufficient restriction or oversight.
- No on-chain validation exists to prevent malicious or unknown modules from being executed.

**Recommendations**

1. Protocol-Level Module Whitelist
   - Enforce an on-chain whitelist of approved modules.
   - Intents referencing modules not in this list must fail validation.
   - This ensures that only audited, protocol-approved logic can interact with smart accounts.
2. Optional User-Level Whitelist
   - Allow each user to maintain their own on-chain "trusted module list."
   - This enables flexibility — users can approve new modules while staying within a globally safe baseline (the protocol whitelist).
3. Explicit On-Chain Authorization for New Modules
   - Adding a new module to either whitelist must require a direct owner action (e.g., a signed on-chain transaction or minting an approval token).
   - Off-chain or UI-only approvals must be rejected.
   - Future improvement: introduce a governance model to manage the protocol whitelist transparently via on-chain voting.
4. Wallet / UX Safeguards
   - Wallets should display both the module hash and verified name/audit reference when signing intents.
   - If the module is not whitelisted, the wallet must warn or block the signing operation.
5. Audited Registry (Optional Enhancement)
   - A public registry or NFT-based system can optionally be used to track audited modules and simplify verification.

**Summary**

This vulnerability enables complete account takeover via malicious modules or compromised TemporaryOwners. Implementing on-chain module whitelisting, explicit authoriza-

tion flows, and wallet-level safeguards is essential to prevent total asset loss and restore user trust in module execution.

**Resolution**

Acknowledged — on-chain whitelists not implemented at this time.

Client rationale: high maintenance/complexity with limited threat-reduction (malicious outcomes achievable via params on existing modules). Current focus is UX/signing clarity (CIP-8 Pondora message prefix) and wallet-side highlighting of Pondora validator addresses for on-chain intents, with longer-term exploration of permissioned sequencing under the open-source/DORA roadmap.

## [M 302] Temporary Owner Delegation Has No Revocation Path

| Level | Severity | Status |
|---|---|---|
| 3 | Medium | Acknowledged |

**Severity Rationale:** High Impact, Low Likelihood

**Category:** Authorization / Key Delegation

**Affected:** TemporaryOwner delegation, off-chain intent authorization, Intent Validator variants

### Description

A TemporaryOwner lets the real owner delegate owner-level authorization (e.g., to a browser session) until `valid_until`. If the `temporary_private_key` is compromised, the user changes their mind, or `valid_until` was entered incorrectly, there is no on-chain way to revoke the delegation. The protocol currently relies solely on `valid_until` — a single point of failure.

### Impact

- Irrevocable exposure until expiry: An attacker with the temp key can continue authorizing intents until `valid_until`.
- Operational risk: Typos or lax expiries cannot be corrected on-chain.
- Incident response gap: No immediate shut-off after suspected compromise.
- Total funds loss: An attacker with the temporary key can drain the smart account — the user can lose all funds.

### Exploit Scenario (illustrative)

1. User delegates a TemporaryOwner for a 12-hour session.
2. The temp key leaks from the browser profile.
3. User realizes the issue but has no way to revoke; attacker can act until `valid_until`.

### Recommendation

1. Publish delegations on-chain (revocable registry):
   - Store each `TemporaryOwner` as an on-chain record (e.g., UTxO + auth NFT, analogous to root-hash publication).
     - The record's live state = delegated
     - absent/burned = revoked.
2. Reference registry during validation:
   - During transaction validation, require a matching live `TemporaryOwner` record for any authorization that relies on a temporary key.
   - Reject if missing, expired, or flagged revoked.
3. Add explicit revoke path:
   - Owner-driven burn: Owner (or a designated recovery key) can burn the delegation NFT / consume the delegation UTxO to revoke immediately.

- Optional “disabled” flag: Support a boolean in the delegation datum to soft-disable without burn (useful for audits/history).

4. Tighten delegation fields:
   - Enforce short maximum validity period for `valid_until` (protocol cap).
5. UX defaults & safeguards (off-chain):
   - Default to very short validity (e.g., minutes–hours).
   - Prominently expose a “Revoke Now” action that constructs the on-chain revoke tx.
   - Warn when `valid_until` exceeds policy caps.

**Resolution**

Acknowledged — on-chain delegation registry/revocation not adopted to preserve instant enablement with per-use keys; users may choose longer validity. Feature remains opt-in with explicit risk disclaimer and a short (≈1 h) default in UX, while keeping on-chain limits flexible.

## [M 303] Unclaimable Protocol Fees from Long-Lived Nonce Intents

| Level | Severity | Status |
|---|---|---|
| 3 | Medium | Acknowledged |

**Severity Rationale:** Medium Impact, High Likelihood

**Category:** Economic Design / Parameter Misconfiguration

**File:** `validators/modules/helper_nonce.ak`

**Description**

In the nonce helper validator, the `protocol_fee_owner` can only reclaim deposited fees by executing `UnregisterCredential` after `intent_valid_until`. If an intent specifies a very distant validity horizon, this effectively time-locks the protocol fee for an impractical duration — potentially years or decades.

Since there is no cap on `intent_valid_until`, a malicious or careless user could issue an intent with a timestamp far in the future, locking protocol fees indefinitely.

**Impact**

- Locked protocol fees: Fees tied to credentials with extreme `intent_valid_until` values become economically unrecoverable until that time passes.
- Persistent revenue leakage: Protocol accumulates stranded funds across multiple such registrations, degrading fee management and accounting accuracy.
- Reduced incentive alignment: Third-party executors may be disincentivized from facilitating intent execution if fees cannot be efficiently reclaimed or recycled.
- Operational risk: Long-lived or abandoned nonce credentials increase state clutter and unproductive locked value on-chain.

**Recommendation**

Redesign the fee flow to pay protocol fees upfront during registration, retaining only a minimal deposit (> 0) to preserve the “single-use nonce” semantics.

- Replace `protocol_fee_owner: VerificationKeyHash` with `protocol_fee_address: Address`.
- On `RegisterAndDelegateCredential`, enforce:
  1. Payment: an output to `protocol_fee_address` with at least `protocol_fee` lovelace in the same tx.
  2. Minimal deposit: `deposit >= 1` lovelace.
- On `UnregisterCredential`, drop the signer check for `protocol_fee_owner`; only require `validity_range > intent_valid_until`.

This eliminates the long-tail liveness dependency for fee recovery and simplifies the flow.

### Severity Rationale

- Impact: Medium — Protocol revenue can be locked indefinitely, leading to economic inefficiency and accounting drift.
- Likelihood: High — Any user can set an extreme `intent_valid_until` without restriction.
- Result: Medium severity classification.

### Summary

If `intent_valid_until` is unrealistically high, the protocol fee becomes time-locked and unrecoverable for extended periods. By paying the fee upfront during registration and introducing a minimal non-zero deposit, the design preserves one-time nonce semantics while eliminating locked protocol revenue and simplifying the unregistration flow.

### Resolution

Acknowledged — protocol retains deferred-fee model

client rationale: off-chain prefiltering lets operators ignore unprofitable intents, upfront fee checks increase execution cost and reduce batching capacity, and operators often self-fund fees when profitable, making delayed fee recovery acceptable.

## [M 304] LabelOutRef Wildcard Intent Theft & Risks

| Level | Severity | Status |
|---|---|---|
| 3 | Medium | Acknowledged |

### Description

The `LabelOutRef` label type allows asset quantities of the associated UTxO to be considered as zero. It is intended explicitly for UTxOs with NFTs or asset bundles. Existing modules don't support NFTs/bundles and so cannot be safely associated with `LabelOutRef`. Doing so would misguide their change calculation (due to `quantity == 0`) and allow all `LabelOutRef` associated UTxOs' assets to be stolen.

Wildcard intents exacerbate this concern, as they can be applied to UTxOs regardless of their label type. A wildcard intent not provisioning for `LabelOutRef`'s zero quantity constraint will essentially allow any account UTxO without `PondDatum` to be spent with it and have the funds stolen.

While protocol's off-chain logic would take care of avoiding this, it is every on-chain module validator's responsibility to only allow compatible labels.

### Recommendation

In each module's (defragment, dependencies, and payments) `if module == own_hash` conditional block, label type should be checked (`label_type != LabelOutRef`). Additionally, if business logic permits, pond variants (`pond_native`) should check that `LabelOutRef` is not used by wildcard intents. Otherwise, every wildcard intent module should be implemented with this potential vulnerability in mind, as it is error-prone.

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

### Resolution

Acknowledged - client preferred mitigation via clear documentation of when wildcard and LabelOutRef intents should be used, as both of these are exceptions rather than the rule.

## [M 305] DDOS of Smart Account Funds via Defragmentation

| Level | Severity | Status |
|---|---|---|
| 3 | Medium | Acknowledged |

### Description

Users with auto-defragmentation-enabled smart accounts are at risk of having their funds perpetually spent by malicious actors. The attack is possible by simply paying transaction fees and executing blocks that contain crucial and time-sensitive intents, resulting in financial losses or missed opportunities for the user. It would also block smart account withdrawals by the user until they manually turn off defragmentation of their funds.

### Attack Methodology

1. Attacker sends `x` UTxOs with min ADA to a smart account with a particular owner's stake credential and `PondDatum {label: datum_label, quantity: datum_quantity, .. }` as its datum with the following attributes:
   - `x + 1 >= min_inputs` to satisfy the minimum required number of inputs for defragmentation
   - `datum_label` to match the label of user UTxOs that need to be spent
   - `datum_quantity == 1` quantity to 1 lovelace for minimal funds lost by attacker and obtained by owner
2. Attacker can now spend the required users' UTxOs continuously by invoking the `module_defragment` and simultaneously creating `x` UTxOs at the user's smart account address.
3. Attacker loses `x` lovelaces (instead of the entire min ADA) in addition to the transaction's fees because the `datum_quantity` is trusted to be accurate. Thus, the cost of executing the DDoS attack is minimal.

Note: Should the UTxOs being defragmented contain FTs, the attacker would lose `x` FTs per DDOS transaction. This cost would still be negligible if the token has greater than three decimal places.

### Recommendation

Multi-Pronged Approach

1. ADA Floor (LabelAda)

For UTxOs with `LabelAda`, a minimum threshold (> 840_000, as no UTxO w/o datum/FTs can have lovelaces lower than this min ada) on quantity can be set to increase attack cost substantially.

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

2. Unit Floor & Batcher Policy (LabelUnit)

- For UTxOs with `LabelUnit`, the ability to take min ADA from all defragmented UTxOs (funds batchers facilitating this service):

    - As a result, lovelaces cannot be returned to the account to make DDOS expensive.
- Additionally, setting a minimum threshold on `datum_quantity` to allow defragmentation is challenging, as it depends on the number of decimals a fungible token has and its price.
    - As the majority of CNTs have six decimal places, a minimum quantity of `1_000_000` can be safely placed for `LabelUnit` UTxOs.
    - While this would exclude defragmentation of FTs with lower or no decimal places, a separate defragmentation module can be created solely for these FTs, which have no minimum quantity requirement. Should accounts really require their defragmentation, they could opt for this specialized defragmentation module.

**Resolution**

Acknowledged — considering current modules are not critically impacted and revocation of defragmentation intent is available as a mitigation strategy. Post migration to DORA, this defragmentation mechanism is scheduled for re-evaluation and modification to demand specialized permissioned environment.

## [L 201] Optimize Defragment Module's Redemptions

| Level | Severity | Status |
|---|---|---|
| 2 | Low | Resolved |

**Category:** Economic Integrity / Cross-Module Composition

**File:** `validators/modules/module_defragment.ak`

### Description

If a pond UTxO is being spent by a defragmentation redemption (intent), there is no need for any other redemptions to be associated with this input, i.e. `length(redemptions) == 1`. The current implementation allows multiple redemptions, increasing complexity and computational cost, and it also enables using the defragment module alongside the payments module to obtain additional fees by invoking the dependencies module.

### Recommendation

Simplify `validate_continuing_redeemers_and_inputs` in `module_defragment` by expecting that a transaction invoking defragmentation does not execute any other modules. This enables efficient validation for every `PondRedeemer { redemptions, .. }` instance by requiring a single redemption solely for the defragment module.

### Resolution

Resolved in commit hash `8a67a19c4761ed2d1933e682f00cc075630c73ba`

## [L 202] Duplicate Root Hash Breaks Intent Revocation Guarantees

| Level | Severity | Status |
|---|---|---|
| 2 | Low | Mitigated |

**Category:** State Consistency / Root Management

**Affected Components:** Root publication & consumption (Intent MPT root lifecycle)

**Description**

Multiple on-chain copies of the same (or different) intent-set root can coexist. When a user "removes" an intent, they typically update or replace one root. However, if any other live root still includes that intent, validators that accept *any matching root* will continue to authorize it — meaning intent revocation is not guaranteed.

**Ways duplicates arise:**

- The "root token" minting policy does not enforce `quantity == 1`, so a user (or attacker tricking the user), or TemporaryOwner can mint multiple tokens in a single tx.
- A User or TemporaryOwner can also mint duplicate tokens across multiple transactions.
- Multiple tokens can carry the same root hash (or different roots that both contain the same member), so revocation in one place doesn't necessarily revoke elsewhere.

**Impact**

- Revocation failure: stale/duplicate roots keep intents executable after "cancellation."
- Amplification by TemporaryOwner: duplicate minting multiplies the problem.
- UX drift: wallet/indexer may show "removed," while chain still accepts proofs.

**Conditions**

- Multiple authoritative sources can exist.
- No per-member tombstone / kill switch is enforced.

**Remediation Plan (Multi-Layer Approach)**

**A) On-Chain Hardening**

**A1. Minting policy — enforce exactly one root token per mint**

- At `validators/intents_native.ak`: reject if minted quantity > 1.

> Outcome: No multi-mint in one tx; reduced duplicate surface across txs.

**A2. Multi-root support, but still authoritative**

Have the ability to store multiple root hashes in the same token/UTxO — this way, the user doesn't need to create multiple tokens, and it becomes more efficient since the required min ADA is reduced when grouping all root hashes within a single UTxO. This design also makes it easier to detect duplicates and to enforce intent removal across all root hashes when requested by the user.

- Store multiple roots in one place: Use `active_roots: Set<RootHash>` in the registry UTxO instead of distributing them across multiple UTxOs.
- Maintain a deduplicated set: Keep an in-datum set of unique root hashes. When cancelling a member *M*, the new registry datum must ensure that *every* root either excludes *M* or, if desired, that *M* is removed only from one specific root (explicitly controlled by the update logic).

> The registry UTxO remains the single source of truth.

> Outcome: > > - Reduced min-ADA footprint. > - Simplified duplicate detection under one UTxO. > - Harder for an attacker to create duplicate roots, since the user mints only once and thereafter only updates roots through the Spend path rather than repeated minting.

**A3. TemporaryOwner limits**

- Disallow TemporaryOwner from minting (allow spend-only)

**A4. Versioning: timestamps**

- Add `version` and `updated_at` fields to the registry.
- Increment `version` and update `updated_at` on every registry change.
- Off-chain logic always uses the registry entry with the latest `version` or `updated_at`.

**B) Off-chain controls**

Handle duplicate roots and user awareness at the interface level. Continuously scan for anomalies, inform the user, and offer remediation actions.

1. Global duplicate scan:
   - Identify all root hashes belonging to the user.
   - Detect identical root hashes across multiple tokens or UTxOs.
   - Detect duplicate intent members across active roots.
2. Alerts:
   - “Duplicate root discovered.”
   - “Member exists in ≥2 roots.”
   - “TemporaryOwner attempted to publish duplicate root hashes (or attempted to mint if A3 is implemented).”
   - “Malicious intents detected.”
3. One-click cleanup:
   - Remove duplicate root hashes.
   - Consolidate multiple roots into a single fresh root, eliminating duplicate intents.

**C) Alternative centralized model (optional)**

Central authorization NFT as staking key Use a user-authorization NFT (policy ID) as the staking key and store it in a dedicated User-Authorization UTxO. This UTxO holds the owner's keys and a boolean flag that acts as an on/off switch for minting root tokens. Each minting or burning transaction must reference this UTxO and toggle the switch accordingly.

- Pros: Provides a single, hard-coded point of authority.
- Cons: Increases system complexity and on-chain cost.

**Resolution**

Mitigated (off-chain) — indexer/UI surface duplicate intents, consolidate duplicate roots on updates, and scan/alert for anomalies; on-chain changes were not adopted. Client opts for flexibility (allow duplicate roots/intents and TemporaryOwner minting)

## [L 203] Unverified `label` / `quantity` in ByModule (`LabelDatum`)

| Level | Severity | Status |
|---|---|---|
| 2 | Low | Mitigated |

**Severity Rationale:** Medium potential impact (future-module misuse), Low likelihood

**Category:** Data Integrity / Validation Assumptions

**File:** `validators/pond_native.ak`

**Scope:** `validate_continuing_redeemers_and_inputs` → `ByModule` → `label_type == LabelDatum`

### Description

In the `LabelDatum` branch of the `pond_native` validator, the redeemer's `label` and `quantity` are only compared against the inline `PondDatum`:

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

There is no validation linking these fields to the actual UTxO `value`. This means a UTxO could technically carry a mismatched datum—for example, a datum that claims a `quantity` of 200 units while the `value` actually contains only 100.

Existing modules are unlikely to exploit this, as they perform their own balance checks. The current design relies on the modules' own logic and economic incentives, which indirectly ensure correctness.

However, future modules could mistakenly rely on these unverified fields and pass validation, leading to incorrect or unbalanced spends, and opening the door for potential exploits if a module assumes these fields are authoritative.

### Impact

- Data integrity risk: Future modules could accept inconsistent state if they treat `ByModule` -exposed `label` and `quantity` as authoritative.
- Potential value mismatch: A malicious or malformed transaction might carry a datum claiming an inflated quantity relative to the actual tokens in the `value`.
- Indirect vulnerability propagation: The issue doesn't currently cause incorrect validation in existing modules but creates a weak trust boundary that could surface in future extensions.

### Recommendations

1. Mitigations
   - Off-chain validation: During transaction building or indexing, verify the correctness of both the datum and the ByModule fields by recomputing the expected `label` and `quantity` from the actual UTxO `value`, and ensuring all three match (`redeemer ↔ datum ↔ value`). Reject or flag any mismatch.
   - Documentation: In the types module, clearly state that `label` and `quantity` at ByModule are not guaranteed to be correct, and are only optimization hints for efficiency.

*Although some guidance already exists in the types module, it should be emphasized that these fields are not guaranteed to be correct and must always be validated within module logic where relevant.*

2. On-chain validation (not preferred; higher cost)
   - Remove the `LabelDatum` branch and stop reading inline datums in `validate_continuing_redeemers_and_inputs` (drop the `datum_label/datum_quantity` pre-read).
   - Always derive `label` / `quantity` from the UTxO `value` using the strict paths (`LabelUnit`, `LabelAda`, `LabelOutRef`).
   - This guarantees correctness centrally and removes any need for later verification at the module level; it also slightly improves efficiency for the remaining branches by avoiding datum access.
   - Trade-off: eliminates the datum-based optimization (making `LabelDatum` redundant) and increases execution cost where that fast path was previously beneficial.
   - It should therefore be considered only if strict on-chain correctness is required, or if the protocol design allows additional budget for higher cost.

**Notes**

- This is not a current exploit in `pond_native` itself; it's a trust-boundary weakness that can surface in new modules if they skip re-checking amounts.
- `LabelUnit` / `LabelAda` already bind quantities to `value`; the gap is specific to `LabelDatum`.

**Summary**

The `LabelDatum` branch does not bind `label` and `quantity` to the UTxO's `value`, creating potential inconsistencies if future modules treat these fields as canonical. Off-chain cross-verification and stronger documentation are some mitigations; full on-chain derivation may be adopted for maximal integrity assurance.

**Resolution**

off-chain validation and documentation clarification applied; on-chain changes were not adopted.
Resolved in commit hash `8a67a19c4761ed2d1933e682f00cc075630c73ba`

## [L 204] Expired Intents Can Be Executed

| Level | Severity | Status |
|---|---|---|
| 2 | Low | Resolved |

**File:** `validators/modules/helper_nonce.ak`

### Description

Nonce Staking Validator is expected to help modules ensure that an intent is executed just once. Once the validity (`intent_valid_until`) has expired, the Staking Validator can be deregistered. Post which, it is possible to re-register it again since its registration (`RegisterAndDelegateCredential: Certificate`) logic fails to check whether the transaction validity range lies before `intent_valid_until`. Should the module relying on this intent not validate the `valid_until` itself (from `Intent`'s `ModuleArgs`), it is possible to execute this intent an infinite number of times by looping through nonce staking validator's uninhibited registration and deregistration.

Only `module_unlock_fungible_outputs_for_payment` of all implemented modules currently relies on the nonce validator, and since it performs required checks on `valid_until`, it remains unaffected. However, a future module can be severely compromised due to the nonce validator's inadequate validation. Relying solely on modules to validate `valid_until` is error-prone and an instance of misplaced responsibility.

### Recommendation

It is expected of a nonce validator to never allow its registration after `intent_valid_until`. Modules can opt in to perform further validations on `valid_until`; however, the lack of it should not enable one-shot intents to be executed more than once after expiry.

### Resolution

Resolved in commit hash `8a67a19c4761ed2d1933e682f00cc075630c73ba`

## [I 101] Use Scott Encoding

| Level | Severity | Status |
|---|---|---|
| 1 | Informational | Resolved |

**Category:** Optimization

**File:** `lib/pondora/pond_native.ak`

### Description

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

We found instances of `validate_continuing_redeemers_and_inputs` utility function where on-chain implemenation could be made much more efficient using Scott Encoding (Back-passing). A trivial example demonstrating performance gains from Scott Encoding:

```txt
┌─ backpassing ──────────────────────────────────
│ PASS [mem: 6_357, cpu: 1_890_943] no_backpassing
│ PASS [mem:   200, cpu:    16_100] with_backpassing
└──────────────────────── 2 tests | 2 passed | 0 failed
```

```Aiken
fn computation_without_backpassing(a: Int, b: Int) -> (Int, Int, Int) {
  (a + b, a * a, b * b)
}

fn computation_with_backpassing(
  a: Int,
  b: Int,
  return: fn(Int, Int, Int) -> any,
) -> any {
  return(a + b, a * a, b * b)
}

test no_backpassing() {
  let (sum, square_a, square_b) = computation_without_backpassing(2, 3)

  18 == sum + square_a + square_b
}

test with_backpassing() {
  let sum, square_a, square_b <- computation_with_backpassing(2, 3)

  18 == sum + square_a + square_b
}
```

### Recommendation

It is recommended to use Scott encoding with `validate_continuing_redeemers_and_inputs` in all the validators.

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

Additional instances where Scott Encoding could be applied:

1. In `module_defragment.ak` line 108:
   - `let new_change_dict, new_change_redemptions_dict <- get_new_change_and_redemptions`
2. In `module_dependencies.ak` line 67:
   - `let new_modules_required, new_modules_observed <- redemptions`
3. In `module_unlock_fungible_outputs_for_payments.ak` line 236:
   - `let new_change_dict, new_change_redemptions_dict <- get_new_change_and_redemptions`

**Resolution**

Resolved in commit hash `8a67a19c4761ed2d1933e682f00cc075630c73ba`

## [I 102] Optimize List Iteration

| Level | Severity | Status |
| --- | --- | --- |
| 1 | Informational | Resolved |

**Category:** Optimization

**File:** `lib/pondora/accounts.ak`

### Description

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

It is slightly more efficient to obtain tail list through pattern matching over `builtin.tail_list` on `change_pairs`. Execution units for both implementations can be found below:

```txt
┌─ pondora ──────────────────────────────────
│ PASS [mem:  28.95 K, cpu:   9.00 M] has_tag
│ PASS [mem:  28.35 K, cpu:   8.91 M] has_tag_optimized
└──────────────────────────── 2 tests | 2 passed | 0 failed
```

### Recommendation

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

### Resolution

Resolved in commit hash `8a67a19c4761ed2d1933e682f00cc075630c73ba`

## [I 103] Optimize Dict Iteration

| Level | Severity | Status |
|---|---|---|
| 1 | Informational | Resolved |

**Category:** Optimization

**File:** `lib/pondora/accounts.ak`

### Description

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

In `correct_change_observed` util function, dictionary containing expected change is iterated twice, first to calculate tag and then to filter out zero change outputs. It is possible to perform both the computations in a single iteration.

### Recommendation

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

### Resolution

Resolved in commit hash `8a67a19c4761ed2d1933e682f00cc075630c73ba`

## [I 104] Optimize `ChangeDict` Creation

| Level | Severity | Status |
|---|---|---|
| 1 | Informational | Resolved |

**Category:** Optimization

**File:** `lib/pondora/account.ak`

### Description

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

Modules collect `OutputReference`s associated with a change output address and label in a list. This list is serialized and hashed to create a unique tag for every output. This process (list creation + serialization + hashing) is expensive and can be simplified by using the first output reference alone. Since a spent output reference is always associated with a single change output address and label, hashing it enough to create a unique tag.

### Recommendation

To just collect the first output reference associated with `change_key` and modify `ChangeDict` to use `OutputReference` instead of `List<OutputReference>`.

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

### Resolution

Resolved in commit hash `8a67a19c4761ed2d1933e682f00cc075630c73ba`

## [I 105] Redundant Key Field in `NativeAuth` Structure

| Level | Severity | Status |
|---|---|---|
| 1 | Informational | Resolved |

**Category:** Optimization / Data Efficiency

**File:** `lib/pondora/types.ak`

### Description

The current `NativeAuth` type is defined as:

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

When a `TemporaryOwner` is present, the first field ( `ByteArray` , representing the signer's public key) duplicates data already contained in `TemporaryOwner.temporary_public_key` . This redundancy increases the redeemer size by approximately 34 bytes (≈ 32 bytes key + 2 bytes CBOR overhead) per transaction that uses session-based (temporary) authorization.

Beyond inefficiency, this duplication introduces a potential state inconsistency, where `signer_pk` and `temporary_public_key` could diverge — a case that must be handled by runtime checks instead of being prevented at the type level.

### Impact

- Increased redeemer size: Adds  34 bytes per redeemer when temporary authorization is used.
- Higher transaction fees: Slightly increases on-chain cost due to additional data size.
- Potential inconsistency: Allows mismatched key fields that type design could otherwise prevent.

While this does not affect validation logic or performance meaningfully, it introduces unnecessary data overhead and minor semantic risk.

### Recommendation

1. Refactor `NativeAuth` to Use a Sum Type

Refactor the `NativeAuth` type to use a sum type for representing the signer and remove redundant fields:

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

This structure:

- Reduces redeemer size for temporary-owner cases (  34 bytes saved per redeemer).
- Eliminates duplication by design.
- Prevents mismatched states through type safety.
- Maintains identical runtime complexity, since pattern matching and Ed25519 verification costs remain constant.

2. Update Dependent Code

Modify any function using the old `NativeAuth` tuple structure.

In `lib/pondora/messages.ak`, the `verify_native_auth` function should now destructure the `Signer` variant:

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

This ensures compatibility with the new type while keeping the same validation semantics.

**Rationale for Severity**

This issue is classified as Informational because it does not pose a security or functional risk. However, addressing it would improve data efficiency, type clarity, and long-term maintainability — desirable qualities for high-frequency on-chain constructs like redeemers.

**Resolution**

Resolved in commit hash `8a67a19c4761ed2d1933e682f00cc075630c73ba`

## [I 106] Optimize Iteration Over Sorted Pairs

| Level | Severity | Status |
|---|---|---|
| 1 | Informational | Resolved |

**Category:** Optimization

**Description**

`Pairs<key, value>` obtained from a `Dict` are sorted on key in ascending lexicographic order. Withdrawals are ordered by ascending `Credential`s too. This allows for a more optimized implementation of `must_have_key_and_remove`, specifically for `validate_modules` and `validate_intents` utility functions. Since keys from one sort pairs are looked up in another, non equal keys can be removed.

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

**Recommendation**

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

**Resolution**

Resolved in commit hash `8a67a19c4761ed2d1933e682f00cc075630c73ba`

## [I 107] Redundant Validation in Spend Path During Burn/Mint

| Level | Severity | Status |
|---|---|---|
| 1 | Informational | Resolved |

**Category:** Optimization / Execution Cost

**File:** `validators/intents_native.ak`

**Description**

In the spend entrypoint, the validator always performs the following heavy checks:

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

When the mint quantity for this token is non-zero, these checks become redundant because the mint entrypoint already enforces the same invariants — including creation and burn semantics, owner binding via outputs, and proper `ChildRef` handling.

Running these validations again in `spend` on burn/mint transactions adds unnecessary execution cost without improving safety.

**Recommendation**

1. Get mint quantity for this token:

   *(Extracting asset_name from the UTxO value is more reliable, since the spending redeemer can differ from the minting redeemer.)*

   *Code excerpt omitted at client request to avoid disclosing non-public source code.*

2. Add redeemer validation for *ChildRef* `auth_method`:

   Since the minting and spending redeemers can differ, If the `auth_method` is ChildRef we must ensure the mint path correctly references the same *owner* and *auth_method*. Insert the following lines before bypassing the heavy checks inside the ChildRef block:

   *Code excerpt omitted at client request to avoid disclosing non-public source code.*

3. Skip redundant checks in `spend` when `mint_qty != 0`:

   - Bypass `all_outputs_paid_to_script` and `is_authorized` in those cases.
   - Keep them active only when no minting or burning occurs (`mint_qty == 0`).

**Reference** Patch (Conceptual)

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

> The heavy invariants (`all_outputs_paid_to_script`, `is_authorized`) should remain in the mint entrypoint for transactions with a non-zero net mint. > The spend entrypoint should become lightweight for burn/mint transactions.

**Impact**

- Reduced execution cost: Avoids redundant checks on burn/mint transactions.
- Clearer separation of concerns:

- *Mint path* → handles semantic and ownership rules.
- *Spend path* → focuses on validation when no minting occurs.

- Simpler maintenance: Heavy validation logic is centralized in one place.

**Resolution**

Resolved in commit hash `8a67a19c4761ed2d1933e682f00cc075630c73ba`

## [I 108] Variant-Level `pond_script` Validation Implemented per Module

| Level | Severity | Status |
|---|---|---|
| 1 | Informational | Acknowledged |

**Category:** Validation Scope / Maintainability

**Files:**

- `validators/pond_native.ak`
- `validators/modules/module_defragment.ak`
- `validators/modules/module_dependencies.ak`
- `validators/modules/module_unlock_fungible_outputs_for_payment.ak`

### Description

Each Pond module currently validates that the intent belongs to the correct Pond variant by asserting:

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

While this protects against executing intents under the wrong validator (e.g., intent crafted for a different variant), it distributes a core invariant across modules. Any newly added module must remember to replicate this check, which creates a maintenance and safety risk:

- If a module omits the `pond_script` check, an intent for a different Pond variant could be accepted under an incorrect validator context.
- The check is not module-specific and should not be re-implemented per module.

### Impact

- Redundancy & Fragility: Duplicating a core invariant across modules increases the chance of inconsistent enforcement as the system evolves.
- Safety-by-Design: Centralizing the check prevents future modules from accidentally omitting it, reducing the risk of variant mismatch leading to unintended unlocks or behaviors.
- Maintainability: A single point of truth simplifies review and reduces boilerplate in modules.

### Recommendation

Centralize `pond_script` Validation in the Pond Variant Validator (`pond_native.ak`) and remove redundant checks from modules.

**1. Extend `Intent` with `pond_variant_script`**

> `lib/pondora/types.ak`

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

**2. Add to `pond_native.ak`**

Enforce `pond_variant_script` & `pond_script` invariants while iterating redemptions:

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

**3. Cross-Variant Validation**

Enforce variant-level consistency across all PondRedeemers so that no intent can be grouped under a mismatched variant. This ensures that each PondRedeemer contains only intents that belong to its declared variant, avoiding inconsistent or unsafe unlock conditions.

Proceed via Option A or Option B, depending on your deployment model.

**Option A — Validate Cross-Variant Redeemer Consistency**

Add a secondary validation path in `pond_native.ak` for PondRedeemers that do not belong to the current variant.

When `my_variant != own_hash`, iterate over all carried intents and assert that `intent.pond_variant_script == my_variant`.

This ensures that every PondRedeemer—whether for the current or a different variant—contains intents that correctly declare the same variant it represents. This prevents any intent from being grouped under the wrong variant.

**Implementation:**

- Core validator (`validators/pond_native.ak`): add the cross-variant check in the `else` branch to assert `intent.pond_variant_script == my_variant` for every carried intent (see diff).

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

**Option B — Whitelist + Co-Invocation Proof**

Introduce a variant whitelist and require co-invocation of any foreign variant referenced by a PondRedeemer that doesn't target the current validator. This ensures we only trust known Pond variants—and only when they are actually executed in the same transaction.

**Implementation:**

- Reference Whitelist UTxO (read-only)
  - Maintain a registry UTxO whose datum lists `allowed_variants: Set<ScriptHash>`.
  - Governance updates the list by rotating this UTxO.
  - Access it via `reference_inputs`.
  - Efficiency option: allow the redeemer to pass an index of the whitelist ref_input.
- Core validator (`validators/pond_native.ak`: `fn validate_continuing_redeemers_and_inputs`) For each PondRedeemer where `redeemer.pond_variant != own_hash` (foreign variant path), add:
  - (B1) Whitelist check: `my_variant ∈ whitelist.allowed_variants`.

- (B2) Co-invocation check: `withdrawals` includes credential corresponds to `redeemer.pond_variant`.

**4. Remove from Modules**

Delete the local `expect pond_script == script_hash` statements along with any supporting logic no longer required in the following modules:

- `validators/modules/module_defragment.ak`
- `validators/modules/module_dependencies.ak`
- `validators/modules/module_unlock_fungible_outputs_for_payment.ak`

This ensures all modules inherit the correct variant validation automatically.

*A concise centralized pattern keeps modules focused on module-specific logic only.*

**Resolution**

Acknowledged—client chose to retain per-module `pond_script` checks for simplicity/performance; centralization deferred.

## [I 109] Redundant Range Handling and Comparison Logic in `must_be_before`

| Level | Severity | Status |
|---|---|---|
| 1 | Informational | Resolved |

**Category:** Performance / Code Clarity

**File:** `lib/pondora/utils.ak`

### Description

The helper function `must_be_before` currently performs unnecessary computations when verifying that a transaction's upper validity bound occurs before a specified deadline:

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

Two inefficiencies were identified:

1. Redundant Bound Comparison — The ledger already guarantees that validity ranges are well-formed (`upper_bound ≥ lower_bound`). Using `max(lower, upper)` is therefore unnecessary and can obscure logic errors.
2. Unnecessary Arithmetic Adjustment — The function adjusts the upper bound value (`time - 1`) to account for exclusivity, then performs a uniform `<` comparison. This can be simplified by adjusting the comparison operator itself, avoiding a subtraction and an extra binding.

### Impact

- Slight efficiency improvement by removing an arithmetic operation and redundant computation.
- Improved code clarity and maintainability, making the logic directly reflect the underlying semantic of inclusive vs. exclusive bounds.
- Eliminates risk of masking invalid ranges that should ideally fail early if malformed.

### Recommendation

Simplify the function to use the upper bound directly and encode inclusivity via the comparison operator:

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

This variant is both clearer and marginally more efficient while preserving the original semantics.

### Resolution

Resolved in commit hash `8a67a19c4761ed2d1933e682f00cc075630c73ba`

## [I 110] Redundant `ModuleArgs` Fields and Checks

| Level | Severity | Status |
|---|---|---|
| 1 | Informational | Resolved |

**Category:** Data Modeling / Maintainability

**File:** `validators/modules/module_unlock_fungible_outputs_for_payment.ak`

### Description

`ModuleArgs` repeats fields already present in `Intent`, leading to duplicate sources of truth and equality checks that add no security value.

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

This duplication motivates guards that merely verify both copies match:

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

Such checks do not harden validation; they only assert internal consistency between two local copies, while the canonical values already exist in the `Intent`.

### Impact

- Redundant logic: Equality checks between duplicates provide no additional safety beyond what `Intent` already specifies.
- Drift risk: Maintaining two sources of truth increases the chance of construction-time divergence.
- Larger redeemers & extra decoding: Unnecessary payload inflates data size and parsing work.
- Maintainability cost: More fields and checks to keep aligned across modules and tests.

### Recommendation

1. Remove duplicated fields from `ModuleArgs`:
   - Delete `nonce_script` and `required_quantity_unlocked` from `ModuleArgs`.
2. Use `Intent` as single source of truth:
   - Read `nonce_script` and `required_quantity_unlocked` directly from the `Intent` where needed.
3. Delete redundant equality checks and dead code:
   - Remove any guards whose sole purpose was to compare the duplicated fields.

### Resolution

Resolved in commit hash `8a67a19c4761ed2d1933e682f00cc075630c73ba`

## [I 111] `nonce_script` Should Use `Option<ScriptHash>`

| Level | Severity | Status |
|---|---|---|
| 1 | Informational | Resolved |

**Category:** Type Safety / Validation Logic

**File:** `validators/types.ak`

**Description**

The `Intent` type currently defines `nonce_script` as a mandatory `ScriptHash`:

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

Comments indicate it is optional and only relevant for intents using the nonce helper:

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

This semantic–type mismatch encourages brittle sentinel checks such as:

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

to emulate optionality.

**Impact**

- Type–semantic mismatch: A mandatory `ScriptHash` suggests every intent requires a nonce, which is incorrect.
- Reduced expressiveness: The current type cannot represent the absence of a nonce, forcing developers to encode it implicitly. Using `Option<ScriptHash>` would make this optionality explicit and safer.
- Redundant checks: Forces manual `is_empty` or sentinel-based guards across modules.
- Potential misuse: Other modules may mistakenly assume a valid nonce is always required.

**Recommendation**

Change the field to an explicit option and replace sentinel checks with pattern matches.

1- Update `Intent` definition

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

2- Replace sentinel checks at `module_unlock_fungible_outputs_for_payment.ak`

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

3- Update helper logic at `validators/modules/helper_nonce.ak`

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

**Resolution**

Resolved in commit hash `8a67a19c4761ed2d1933e682f00cc075630c73ba`

## [I 112] Remove Redundant Owner Checks from Modules

| Level | Severity | Status |
|---|---|---|
| 1 | Informational | Resolved |

**Category:** Validation Scope / Maintainability

**Files:** `validators/modules/*`

**Description**

Multiple modules redundantly re-validate ownership using:

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

However, ownership is already enforced centrally within the `pond_native` validator:

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

This makes the per-module `address_owner == owner` check redundant.

**Impact**

- Consistency & Safety: Centralizing ownership checks ensures a single source of truth, eliminating the risk of module-level inconsistencies.
- On-Chain Efficiency: Avoids paying twice for the same invariant.
- Maintainability: Future changes to ownership semantics only need to be implemented once.
- Code Footprint Reduction: Removes repeated boilerplate logic from every module.

**Recommendation**

Delete the redundant `expect address_owner == owner` line and any associated setup logic that exists solely to support this check.

1. Remove duplicate ownership checks from all modules (`validators/modules/*`):

   *Code excerpt omitted at client request to avoid disclosing non-public source code.*

2. Clean up dead code

   Remove any local derivations or unused imports tied to this redundant check:

   - Local stake credential extraction:

     *Code excerpt omitted at client request to avoid disclosing non-public source code.*

   - Unused imports:

     *Code excerpt omitted at client request to avoid disclosing non-public source code.*

This keeps modules focused strictly on module-specific logic, consistent with Pond's design principle that shared validation (like ownership enforcement) belongs in the variant-level validator (`pond_native`).

**Summary**

Ownership is already guaranteed by `pond_native`. Removing per-module owner checks and their supporting logic reduces duplication, ensures consistency, and keeps modules clean and purpose-focused.

**Resolution**
Resolved in commit hash `8a67a19c4761ed2d1933e682f00cc075630c73ba`

## [I 113] Spam UTxO / Token Pollution

| Level | Severity | Status |
|---|---|---|
| 1 | Informational | Mitigated |

**Category:** Input Pollution / UX Integrity

**Affected Components:** Smart account UTxO set, off-chain builders, withdrawal modules

**Description**

An external party can pollute the smart account by:

1. sending UTxOs to the account with bogus / unexpected datums, and/or
2. sending spam tokens — arbitrary assets under foreign policy IDs, potentially containing phishing links or deceptive metadata — to the account address.

While this doesn't grant the attacker spend authority, it can break module assumptions and bloat state, leading to:

- on-chain validation failures (if the off-chain builder doesn't properly handle unexpected datums or values)
- degraded withdrawal experience (user unintentionally receives spam tokens alongside intended assets).

*Key constraints: On-chain, the account cannot reliably "know" which tokens are user-intended vs spam unless an allowlist is explicitly modeled. Anyone can send tokens to any address.*

**Impact**

- Validation friction / soft DoS: Transactions fail if the module expects an exact datum/value layout but encounters polluted inputs.
- Poor UX on withdrawals: Users may receive spam tokens with their assets.

**Scenario (illustrative)**

- Attacker sends many tiny UTxOs with misleading datums to the Pond address. A module that expects a particular datum schema or single-policy `Value` now sees polluted inputs and reverts.
- Attacker airdrops junk tokens to the address. A naïve withdrawal flow that mirrors all assets back to the user returns spam tokens too, surprising the user and increasing fees.

**Recommendation**

1. Off-chain mitigations
   - Validate datums & values before build: Detect any UTxO with an invalid or unexpected datum/value structure and notify the user. The interface should display these UTxOs clearly, allowing the user to decide whether to consume them directly or clean them later using the Garbage Cleaner module.
   - Tag spam tokens:
     - Detect likely spam (e.g., unknown/unsolicited assets, suspicious metadata) and mark them as "Spam" in the UI.

    - Clearly notify the user when spam is present.
    - Keep spam tokens tagged and visible.
  - Explicit user choice: When withdrawing, keep spam tokens visible but require explicit user selection to include them; default flows shouldn't auto-sweep spam with intended assets.

2. Garbage Cleaner module (on-chain)
   - Deploy a Garbage Cleaner module to sanitize the smart account under owner authorization.
   - Handle spam tokens safely:
     - Transfer tagged spam to a designated quarantine address (owner-controlled).
     - Group spam tokens into a labeled "Quarantine" UTxO within the same account, separating them from normal operations.
   - Fix datums: Recreate UTxOs with valid, schema-correct datums so that `LabelDatum`-based logic can operate efficiently and without errors.
   - Consolidate UTxOs: Merge tiny/redundant UTxOs to improve efficiency and execution reliability.

**Resolution**

Mitigated off-chain via indexer filtering—UTxOs with invalid datums or non-whitelisted tokens are ignored; UX prompts and the on-chain Garbage Cleaner were not implemented. Resolved in commit hash `8a67a19c4761ed2d1933e682f00cc075630c73ba`

## [I 114] ChildRef Root Index Optimization

| Level | Severity | Status |
|---|---|---|
| 1 | Informational | Resolved |

**Category:** Performance Optimization / Reference Input Traversal

**File:** `validators/intents_native.ak`

### Description

Each `ChildRef` argument currently carries a full `child_ref_root` value, which must be verified by scanning all reference inputs to locate the matching root. This introduces redundant iteration, higher CPU/memory usage, and unnecessary redeemer bloat while performing a simple lookup operation.

### Recommendation

Replace the `child_ref_root` field with an index-based reference to the correct root input. This allows direct access to the intended reference input without iterating over all of them.

### Proposed Structure

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

### Conceptual Implementation

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

This approach removes redundant traversal, validates directly via the indexed reference input, and still enforces the binding between the owner token and on-chain MPT root.

### Impact

- Reduced execution units / lower fees: Removes O(n) reference input traversal — replaced with a constant-time O(1) lookup.
- Smaller redeemer size: The full `child_ref_root` `ByteArray` no longer needs to be serialized in the redeemer.
- Simpler logic: Canonical reference by index avoids ambiguity and mismatches between redeemer and transaction structure.
- Security preserved: Still validates ownership and MPT membership via `policy_id`, `owner`, and the on-chain root extracted from the reference input.

### Summary

Switching from root-by-value to root-by-index simplifies validation, reduces CPU/memory usage, lowers fees, and eliminates redundant scanning — while maintaining full security guarantees through direct reference input binding.

### Resolution

Resolved in commit hash `8a67a19c4761ed2d1933e682f00cc075630c73ba`

## [I 115] Incorrect Comment on `valid_from` Check

| Level | Severity | Status |
|---|---|---|
| 1 | Informational | Resolved |

**Category:** Code Clarity / Documentation Accuracy

**File:** `validators/modules/module_unlock_fungible_outputs_for_payment.ak`

### Description

The comment above the `valid_from` validation is incorrect. It currently reads:

```
1 // first make sure the valid_from is after the current time
```
Aiken

However, the logic actually ensures that the current time is after `valid_from`, not the reverse.

This inconsistency could cause confusion during audits or future maintenance.

### Recommendation

Update the comment to correctly reflect the logic:

```
1 // make sure the current time is after valid_from
```
Aiken

### Resolution

Resolved in commit hash `8a67a19c4761ed2d1933e682f00cc075630c73ba`

## [I 116] Redundant `owner` Field in Uniqueness Tuple

| Level | Severity | Status |
|---|---|---|
| 1 | Informational | Resolved |

**Category:** Code Optimization / Redundancy

**File:** `validators/modules/helper_nonce.ak`

### Description

In the following section of the nonce helper validator, a `(owner, label, intent)` tuple is serialized to track unique redemptions:

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

However, this `owner` field is redundant. The insertion only occurs when `owner == intent_owner`, which is already enforced by the surrounding logic:

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

Therefore, the `owner` value is constant within this scope and provides no additional uniqueness information.

### Impact

- Minor increase in serialized data size and execution budget consumption.
- Slightly less clarity, as the presence of `owner` may suggest multiple owners could be considered, which is not the case here.

### Recommendation

Simplify the tuple key to only include fields that actually vary:

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

This improves both clarity and efficiency without changing behavior.

### Resolution

Resolved in commit hash `8a67a19c4761ed2d1933e682f00cc075630c73ba`

## [I 117] Miscellaneous Optimizations

| Level | Severity | Status |
|---|---|---|
| 1 | Informational | Resolved |

**Category:** Optimization

**Description**

Following minor optimizations were found:

1. In `intents_native.ak` on line 77, `owner` obtained from deconstructed redeemer can be used instead of `redeemer.owner`.
2. In `intents_native.ak` reference input index containing `child_ref_root`, parent module invoking redeemer's index and corresponding spent input's index can be provided in `ChildRef` for efficient lookups.
3. To avoid an empty list, instead of calculating its length, `list.is_empty` should be used. Inefficient checks observed at following places:
   - In `module_dependencies.ak` on line 88, `expect list.length(required_intents) > 0`
   - In `module_dependencies.ak` on line 202, `dict.size(modules_required) > 0` (`dict.is_empty` to be used)

**Resolution**

Resolved in commit hash `8a67a19c4761ed2d1933e682f00cc075630c73ba`

## [I 118] Optimize Required Intents Serialization

| Level | Severity | Status |
|---|---|---|
| 1 | Informational | Resolved |

**Category:** Optimization

**Description**

Dependencies module provides required intents through `ModuleArgs` (`type ModuleArgs = List<(ByteArray, ByteArray, Intent)>`), every tuple (`(ByteArray, ByteArray, Intent)`) present in the list is directly serialized without requiring any element specific validation.

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

**Recommendation**

It is recommended to keep each required intent element (tuple `(ByteArray, ByteArray, Intent)`) serialized, to eliminate on-chain serialization costs.

*Code excerpt omitted at client request to avoid disclosing non-public source code.*

**Resolution**

Resolved by avoiding `Dependency` serialization altogether.
Resolved in commit hash `df25d371b89fe747563fcc909816f55cdecb98c5`

## End of Report


No Witness Labs — Confidential

Copyright © 2025